

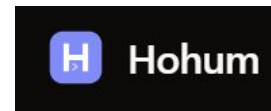
HW1P1 Bootcamp

Ron Sarma, Dinesh Varun, Yara Yao



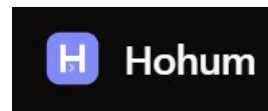
Getting Started

Release



August 28th, 6 PM ET

Completion



~ 3 weeks

Submission

handin.zip



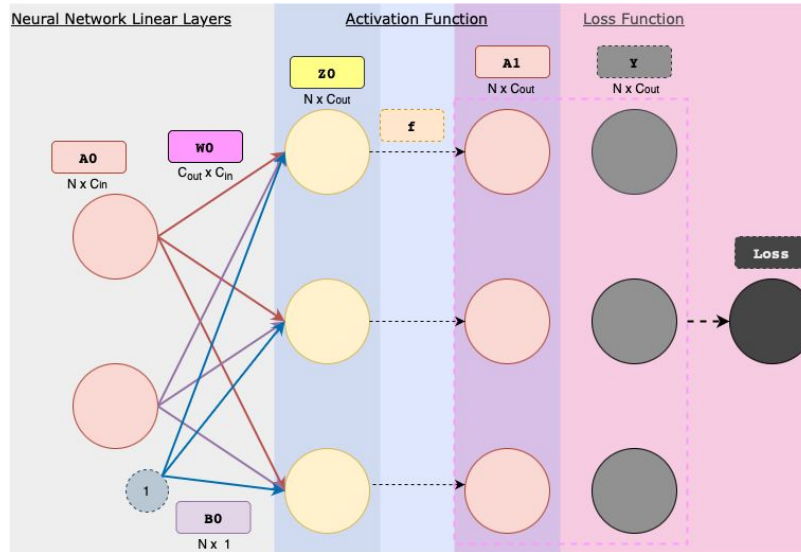
EARLY DDL: Sept 4th, 11:59 PM ET
ON-TIME DDL: Sept 18th, 11:59 PM ET

Objectives & Overview

MyTorch[©]

- Your personalized deep-learning framework, inspired by PyTorch
- Robust – MLP, CNN, RNN, GRU, LSTM, Transformers
- Evolve – Grows with you as the semester progresses

HW1P1 – MLP



- Network Setup
- Forward Pass Functionalities
 - Activation Functions
 - Loss Functions
 - Optimizers
- Backward Pass Functionalities
- Regularization Methods

HW Policies

Collaborative

- Work in Study Groups (not more than 4)
- You are allowed to discuss and share ideas, but not code
- Discuss with your TA mentors

AI Usage

- Use it wisely - maximize learning
- Read the [IDL AI Usage Policy](#) and [Academic Integrity Policies](#)
- Refer to our [Socratic Prompting Guidelines](#)

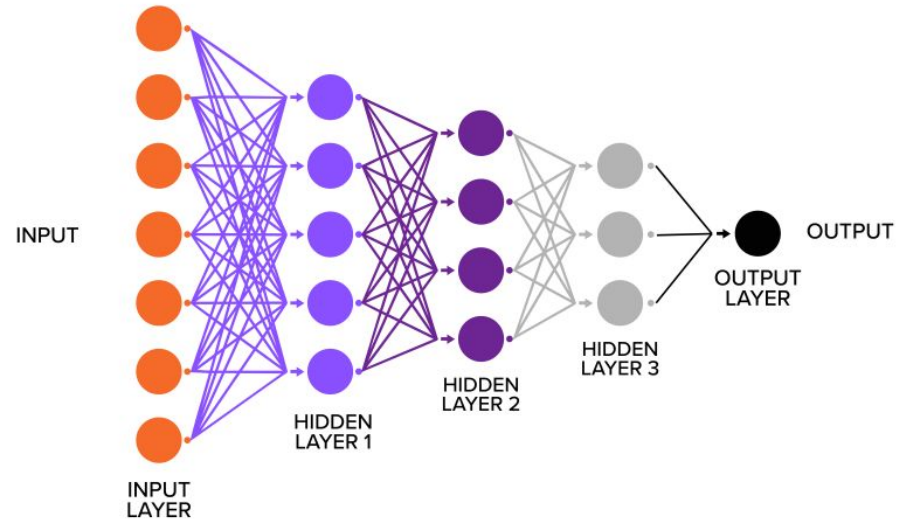
Setup

Anaconda (miniconda) and virtual environment

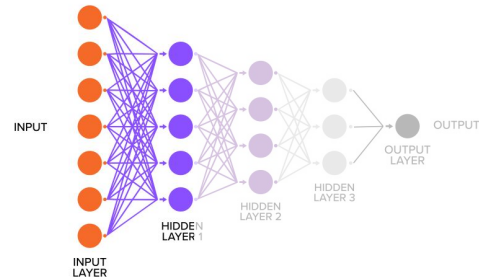
- `cd /path/to/HW1P1`
- `conda create -n idlf26 python=3.13 -y`
- `conda activate idlf26`
- `pip install -r requirements.txt`

```
hw1p1_handout
├── mytorch
│   ├── nn
│   │   ├── linear.py
│   │   ├── activation.py
│   │   ├── loss.py
│   │   └── batchnorm.py
│   └── optim
│       └── sgd.py
├── models
│   └── mlp.py
├── autograder
│   ├── hw1p1_autograder.py
│   └── hw1p1_autograder_flags.py
├── requirements.txt
└── create_submission.py
```

The Neural Network



Linear Layers



The forward pass

$$Z = A \cdot W^T + \iota_N \cdot b^T$$

The backward pass

$$\begin{aligned} \frac{\partial L}{\partial A} &= \left(\frac{\partial L}{\partial Z} \right) \cdot \left(\frac{\partial Z}{\partial A} \right)^T = \left(\frac{\partial L}{\partial Z} \right) \cdot W \\ \frac{\partial L}{\partial W} &= \left(\frac{\partial L}{\partial Z} \right)^T \cdot \left(\frac{\partial Z}{\partial W} \right) = \left(\frac{\partial L}{\partial Z} \right)^T \cdot A \\ \frac{\partial L}{\partial b} &= \left(\frac{\partial L}{\partial Z} \right)^T \cdot \left(\frac{\partial Z}{\partial b} \right) = \left(\frac{\partial L}{\partial Z} \right)^T \cdot \iota_N \end{aligned}$$

Code Name	Math	Type	Shape
N	N	scalar	-
in_features	C_{in}	scalar	-
out_features	C_{out}	scalar	-
A	A	matrix	$N \times C_{in}$
Z	Z	matrix	$N \times C_{out}$
W	W	matrix	$C_{out} \times C_{in}$
b	b	matrix	$C_{out} \times 1$
dLdZ	$\partial L / \partial Z$	matrix	$N \times C_{out}$
dLdA	$\partial L / \partial A$	matrix	$N \times C_{in}$
dLdW	$\partial L / \partial W$	matrix	$C_{out} \times C_{in}$
dLdb	$\partial L / \partial b$	matrix	$C_{out} \times 1$

Activation Functions



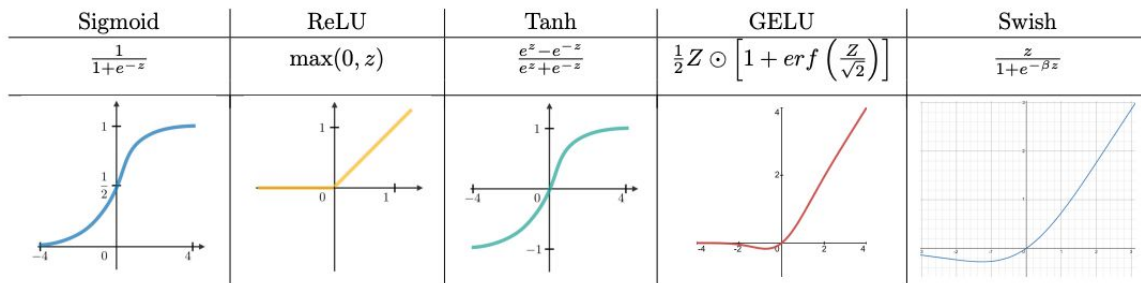
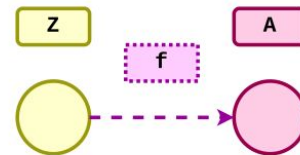
Activation Functions

Why?

- Introduce **Non-Linearity** in the Neural Network to approximate nonlinear functions.
- Without activation functions, network will always be linear, no matter how deep it is !!

Types:

- Scalar Activations
 - Applied element wise
 - e.g. ReLU, Sigmoid, Tanh, GELU, Swish
- Vector Activations
 - Each output element depends on all input elements
 - e.g. Softmax

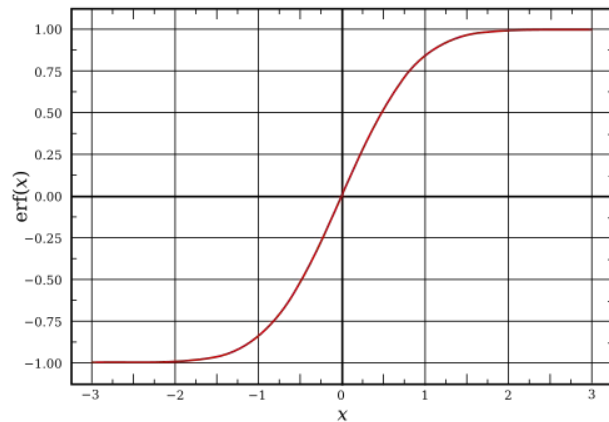


$$a_m = \frac{\exp(z_m)}{\sum_{k=1}^C \exp(z_k)}$$

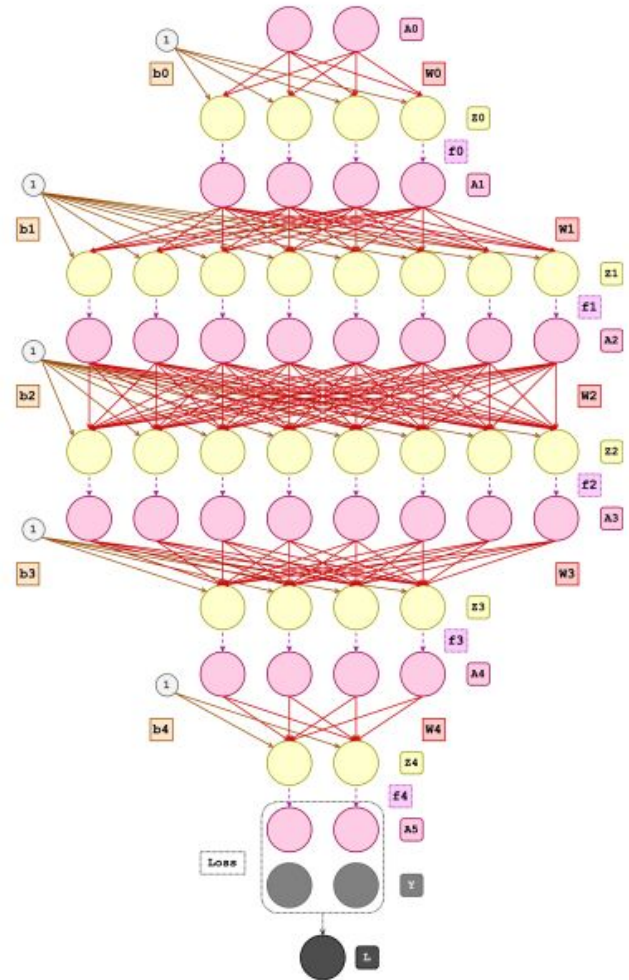
Activation Functions

Hints:

- Read about **NumPy broadcasting**
- Read the docs on `np.amax`, `np.maximum`, `np.where`
- Read about **Error Function** (erf) - search and read the docs of `math` and `scipy` library for help with implementation.
- Read and implement **Numerically Stable Softmax** to address arithmetic overflow
- Code Hints in the Handout.



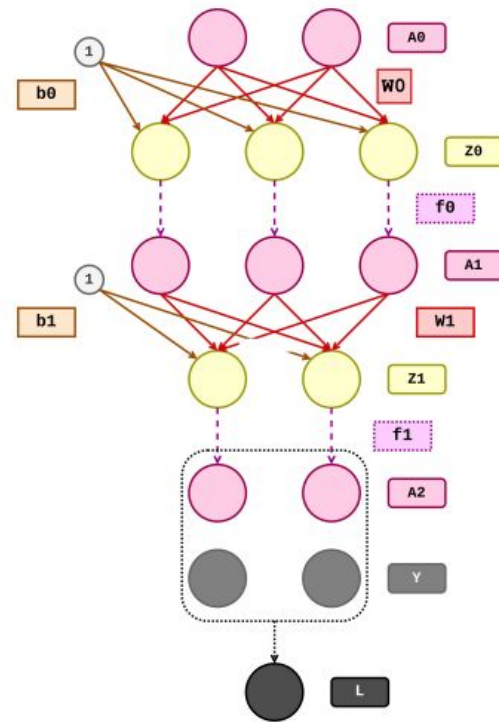
MLP



Building the MLP

Common Pitfalls and Fixes:

- Mistake 1: Treating Linear and Activation Differently
 - Don't write separate logic for Linear vs Activation layers
 - **Solution:** Both are just `self.layers[i]` - call their forward/backward methods the same way
- Mistake 2: Backward in Forward Order
 - Don't loop through layers 0 to end during backward pass
 - **Solution:** MUST use `reversed(range(len(self.layers)))` to go from end to 0
- Mistake 3: Wrong Dimensions
 - Don't guess the input/output sizes for each Linear layer
 - Solution: Draw the network on paper! Follow the arrows in Figures G, H, and I from the writeup



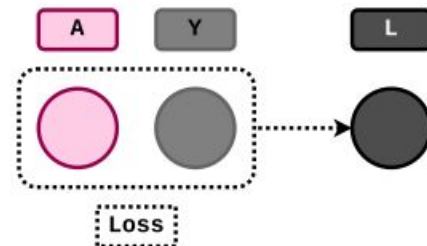
Loss Functions



Loss Functions

Why?

- A loss function measures how wrong your model's predictions are.
 - It outputs a single scalar value that quantifies the mismatch between:
 - Model Output (A): What your network predicted
 - Ground Truth (Y): What the correct answer should be
- Lower loss equals better model! We use this value to update network parameters via optimization.



$$SE(A, Y) = (A - Y) \odot (A - Y)$$

$$SSE(A, Y) = \iota_N^T \cdot SE(A, Y) \cdot \iota_C$$

$$MSELoss(A, Y) = \frac{SSE(A, Y)}{N \cdot C}$$

MSE (Mean Squared Error) Loss

- **Use Case:** Predicting continuous values (House prices, Temperature, Stock price)

Cross-Entropy Loss

- **Use Case:** Predicting categories (cat vs dog, digit 0-9, spam vs not spam)
- Negative log probability of correct class.

$$\begin{aligned} \text{softmax}(A) &= \sigma(A) \\ &= \frac{\exp(A)}{\sum_{j=1}^C \exp(A_{ij})} \end{aligned}$$

$$\begin{aligned} \text{crossentropy} &= H(A, Y) \\ &= (-Y \odot \log(\sigma(A))) \cdot \iota_C \end{aligned}$$

$$\begin{aligned} \text{sum_crossentropy_loss} &:= \iota_N^T \cdot H(A, Y) \\ &= SCE(A, Y) \end{aligned}$$

$$\text{mean_crossentropy_loss} := \frac{SCE(A, Y)}{N}$$

Optimizers



Optimizers

Why?

- Backprop gives us the **gradients** — how loss changes with each parameter.
- The optimizer uses those gradients to update W and b , lowering the loss.

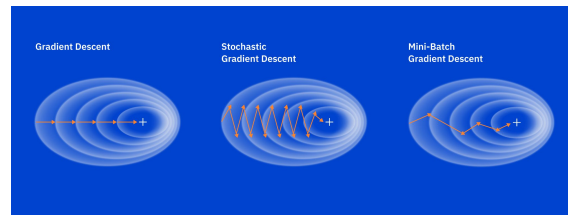
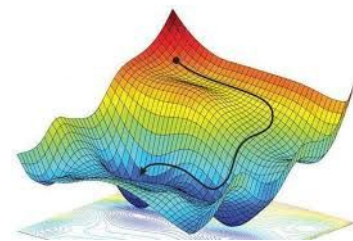
SGD Update Rule:

- Without Momentum
 - Simple, but can be slow or oscillate on noisy gradients

$$W := W - \lambda \frac{\partial L}{\partial W}$$
$$b := b - \lambda \frac{\partial L}{\partial b}$$

- With Momentum

$$v_W := \mu v_W + \frac{\partial L}{\partial W} \quad W := W - \lambda v_W$$
$$v_b := \mu v_b + \frac{\partial L}{\partial b} \quad b := b - \lambda v_b$$



Regularization



Regularization

Why?

- Batch Norm **re-centers and re-scales** each layer's inputs, using statistics computed over the batch.
- This reduces internal covariate shift, making training faster and more stable.

Two Modes:

- Training (eval = False)
 - Normalize using the current batch's mean μ and variance σ^2
 - Update running_M / running_V for later use at inference
- Inference (eval = True)
 - Normalize using the learned running_M / running_V instead
 - Vectorize everything — no for loops over features!

Regularization

$$\mu_j = \frac{1}{N} \sum_{i=1}^N Z_{ij}$$

$$\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (Z_{ij} - \mu_j)^2$$

$$\hat{Z}_i = \frac{Z_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

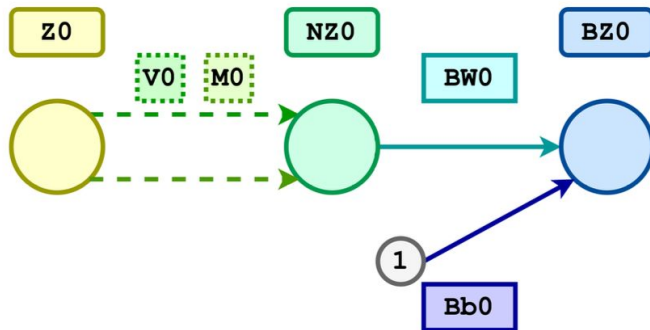


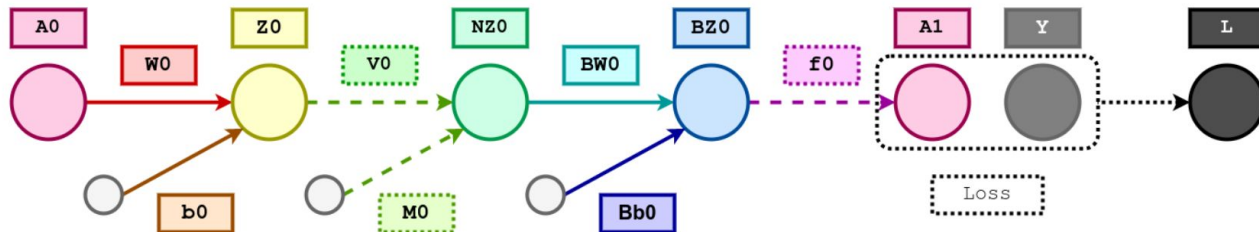
Figure L: Batchnorm Topology

$$E[Z] = \alpha * E[Z] + (1 - \alpha) * \mu$$

$$Var[Z] = \alpha * Var[Z] + (1 - \alpha) * \sigma^2$$

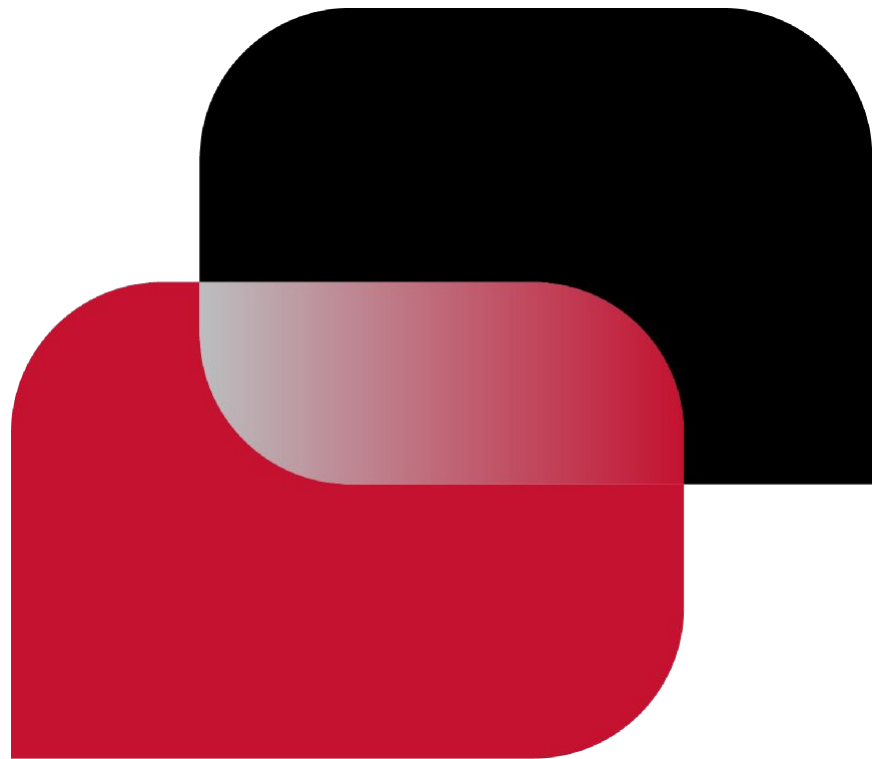
$$\hat{Z}_i = \frac{Z_i - E[Z]}{\sqrt{Var[Z] + \epsilon}}$$

$$\tilde{Z}_i = \gamma \odot \hat{Z}_i + \beta$$



Slide Deck Title

Tagline



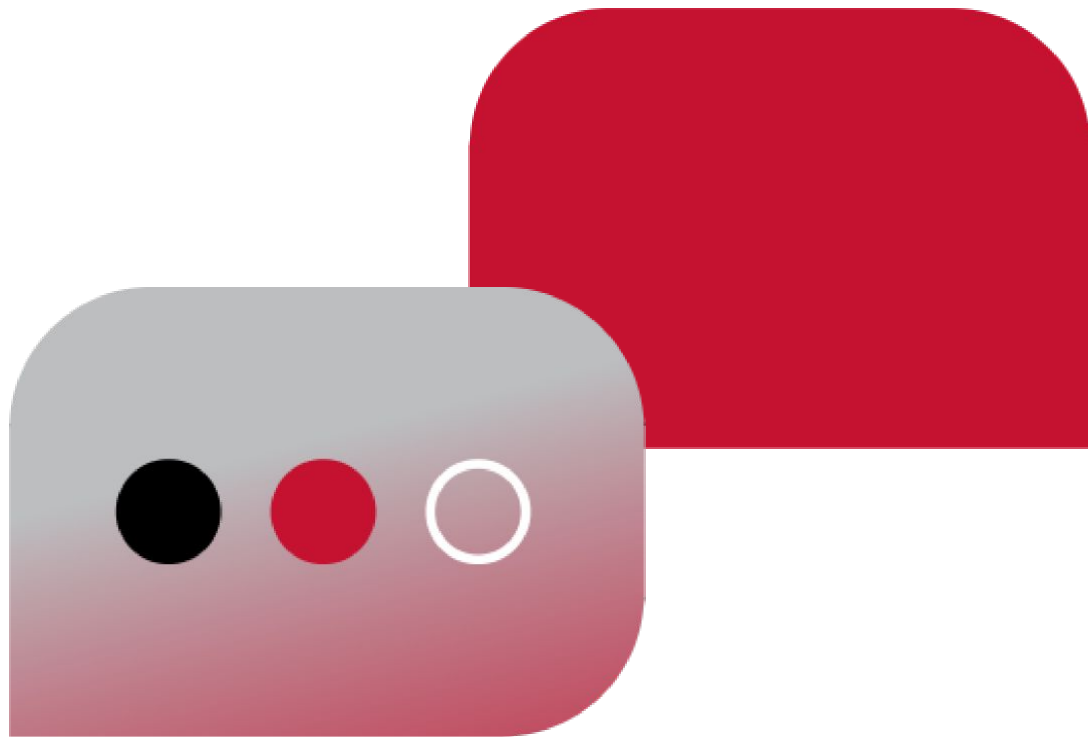


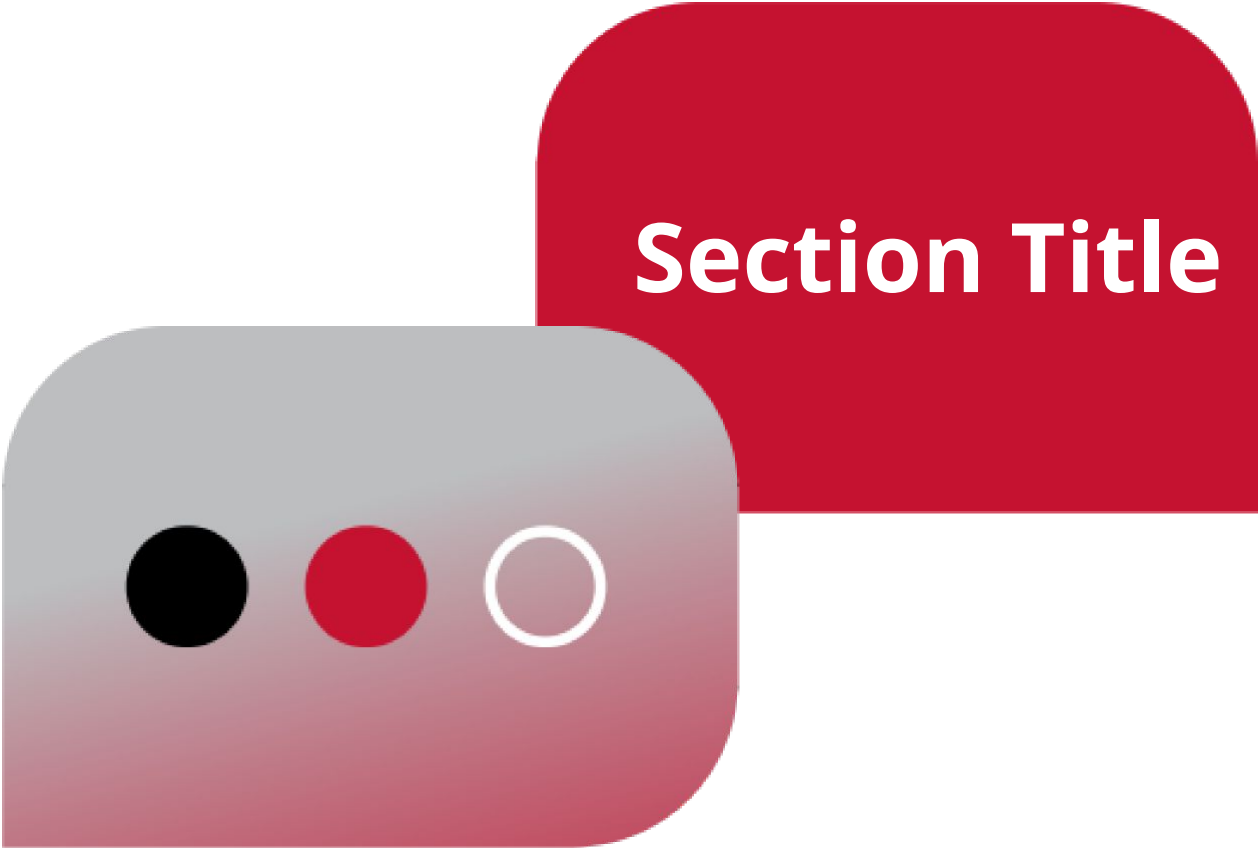
Slide Deck Title

Tagline

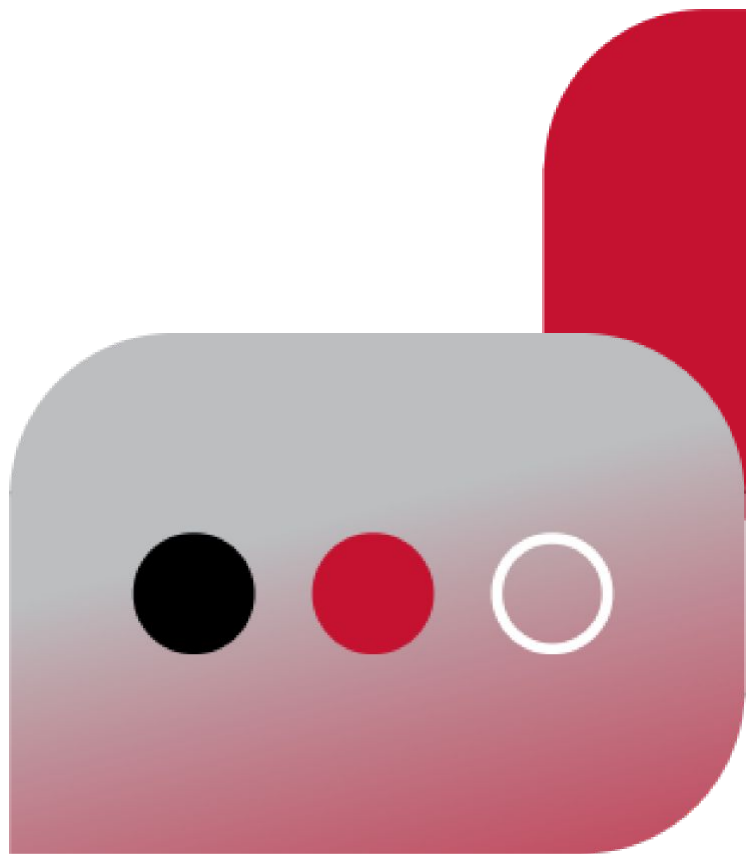
Section Title

Tagline





Section Title



Section Title

Tagline

Title

- Body Text / Bullet Points
- Body Text / Bullet Points
- Body Text / Bullet Points
- Body Text / Bullet Points



Simple list

First thing

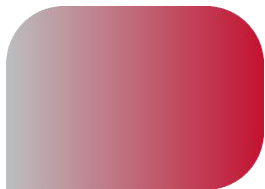
Add a quick description of each thing, with enough context to understand what's up.

Second thing

Keep 'em short and sweet, so they're easy to scan and remember.

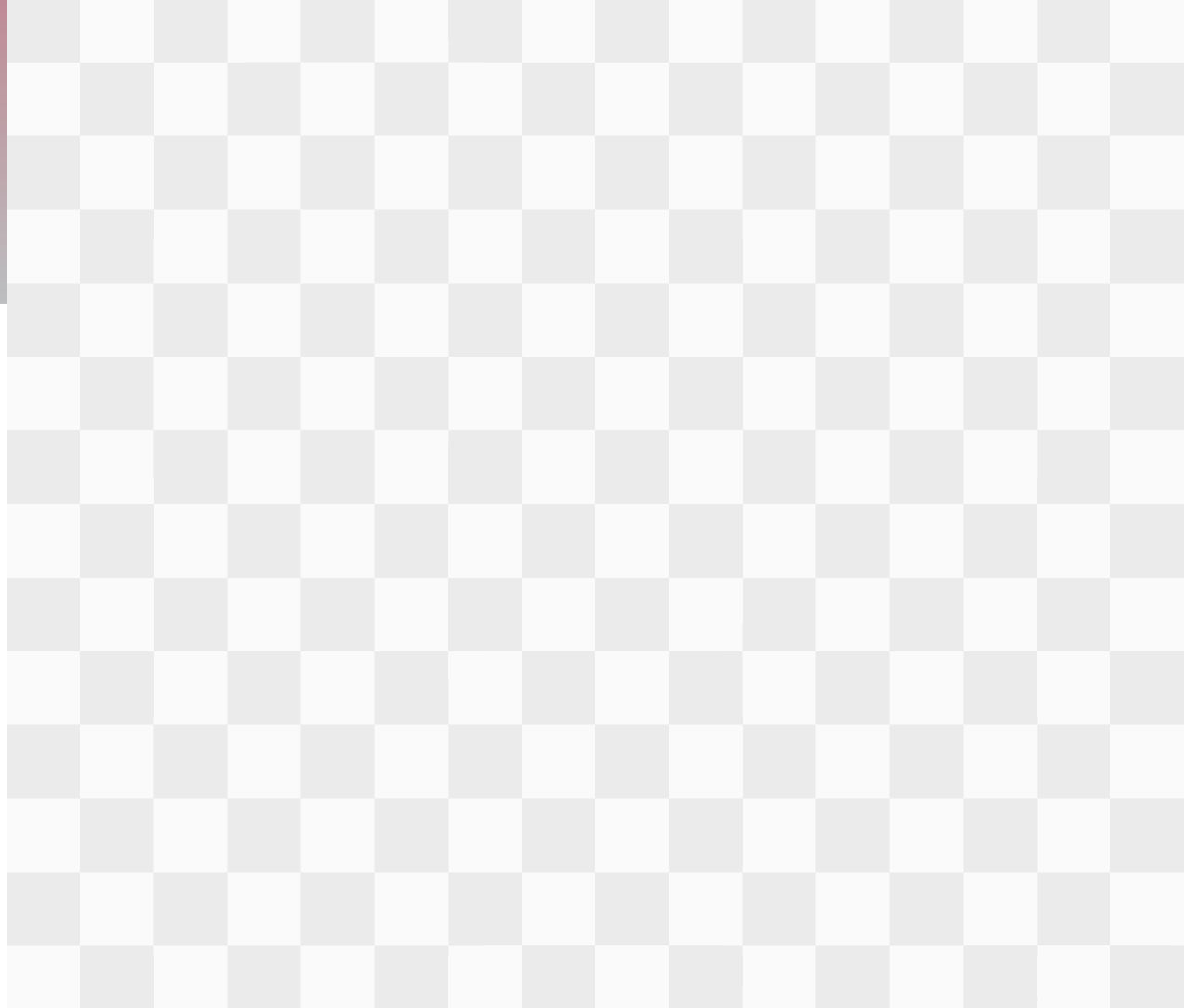
Third thing

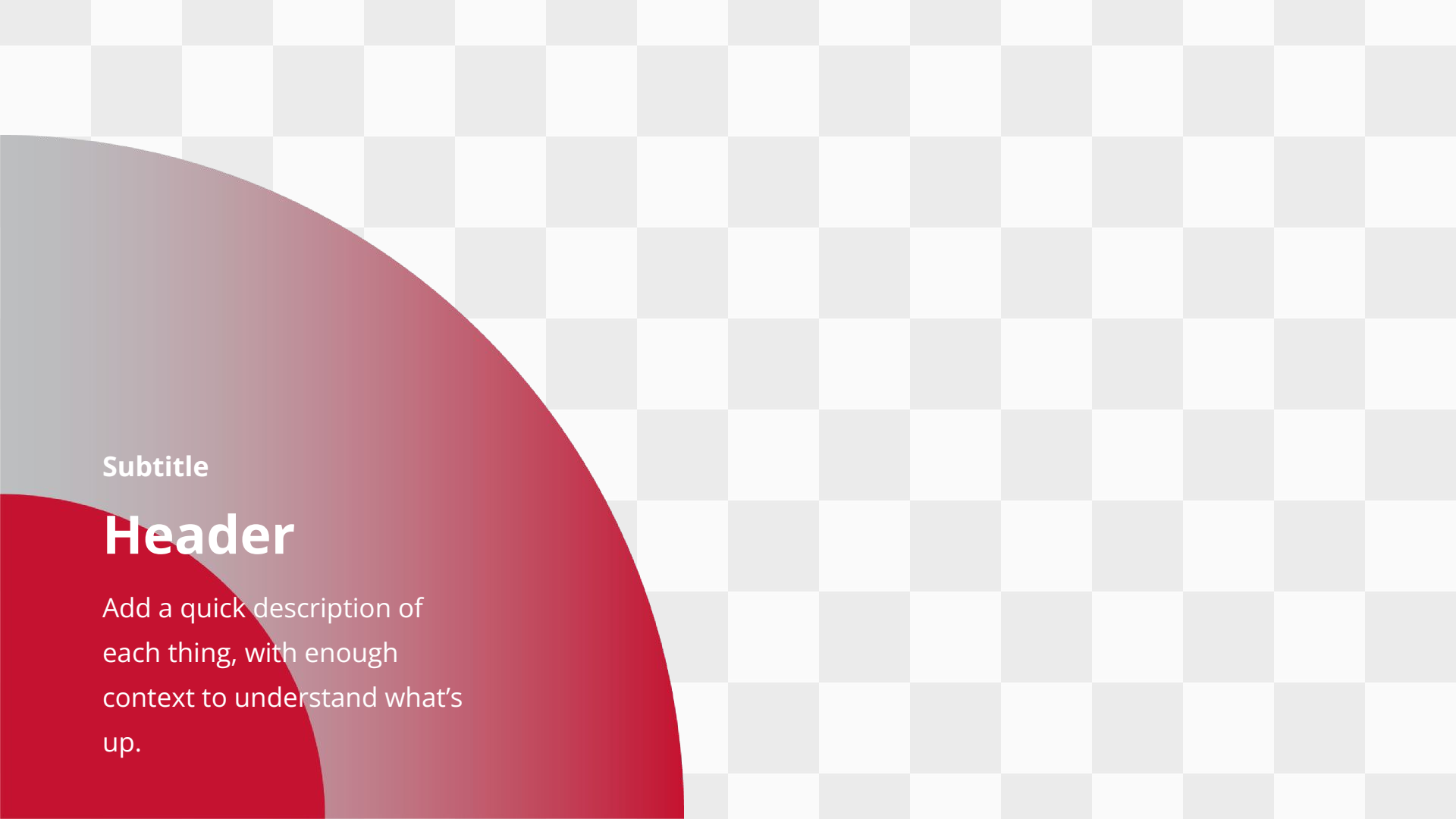
If you've got a bunch, add another row, or use multiple copies of this slide.



Subtitle

Header

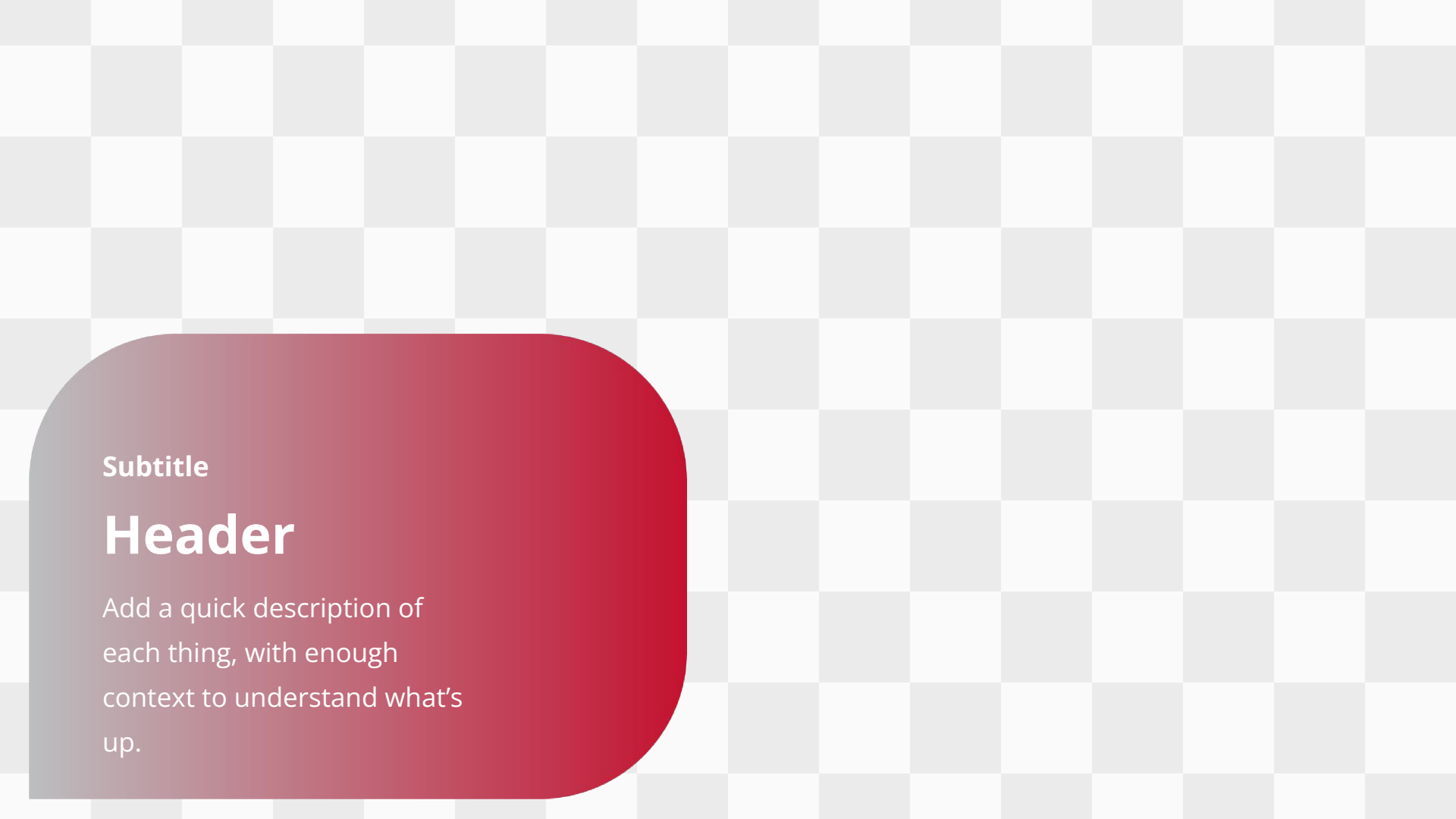




Subtitle

Header

Add a quick description of each thing, with enough context to understand what's up.



Subtitle

Header

Add a quick description of each thing, with enough context to understand what's up.



Use this slide to highlight a single, important thing. To keep it short and sweet, you might link away to relevant doc or file.



Use this slide to highlight a single, important thing. To keep it short and sweet, you might link away to relevant doc or file.